

MODULE 11

GitHub Copilot for Testing and Refactoring

Use AI to generate tests, uncover edge cases, and refactor code — while you stay the final reviewer.





MODULE 11 OVERVIEW

Testing consumes significant developer time — AI can accelerate test creation, refactoring, and code review.

- Building on Module 10's fundamentals, you'll now use Copilot for testing-specific and refactoring-specific workflows — generating tests, finding edge cases, and cleaning up code.

DURATION & LAB



1 Hour Theory

AI-assisted testing, refactoring, code review.



45 Min Lab

AI-Assisted Test Generation.



Scenario

Generate unit tests, edge cases, and mocks for a service.

AI GENERATES

Unit tests, edge cases, mocks, and refactoring suggestions

YOU REVIEW

Correctness, security, quality, and maintainability

THE PAYOFF

Higher coverage, cleaner code, faster delivery

Enterprise scenario: in a large banking application, Copilot helps teams generate high-quality tests, cover edge cases, and refactor legacy code faster.

KEY TAKEAWAY AI is your assistant, not a replacement. Your judgment ensures enterprise-quality tests and code.

WHY TESTING AUTOMATION MATTERS

Automated testing is essential for delivering high-quality software faster, with confidence and efficiency.

WITHOUT TEST AUTOMATION

- Slow and manual — tests delay feedback
- More bugs escape to production
- High maintenance cost over time
- Risky, low-confidence releases

VS

WITH TEST AUTOMATION

- Fast, early feedback on every change
- Better coverage across more scenarios
- Cost-effective — enables CI/CD
- Confident, reliable releases

THE BUSINESS & TECH IMPACT



**Faster
Feedback**



**Higher
Quality**



**More
Productivity**

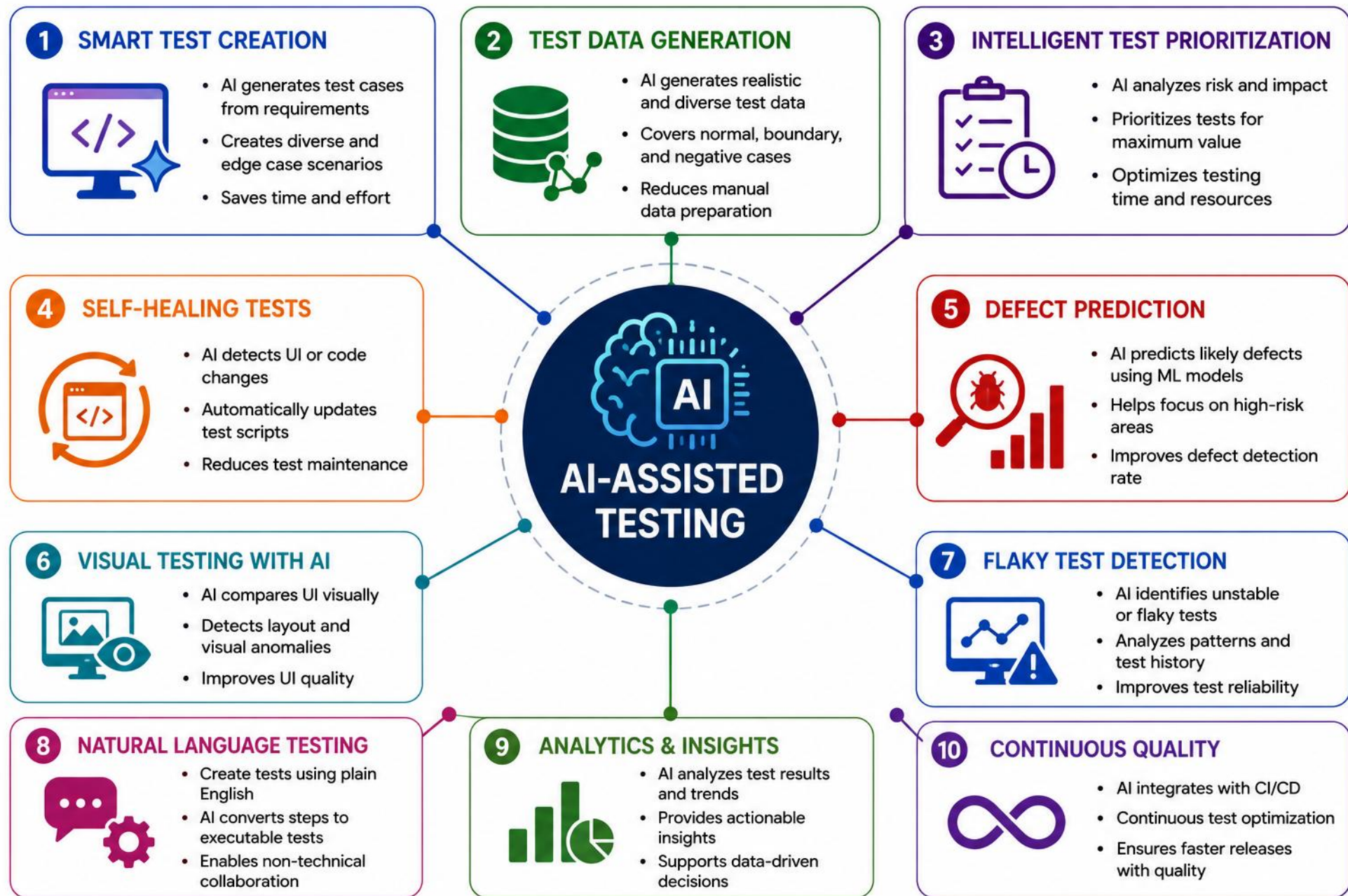


Lower Costs



**Continuous
Delivery**

KEY TAKEAWAY Testing automation isn't just about writing tests — it's about delivering value faster and building software users trust.



HOW COPILOT SUPPORTS TESTING

Six ways Copilot supports your testing work — from generating the first test to requiring your review.

WHAT COPILOT DOES AT EACH STAGE



Generate Tests

Creates unit tests directly from your code and its behavior.



Suggest Edge Cases

Surfaces boundary, negative, and failure scenarios to cover.



Create Mocks

Builds mocks and stubs so dependencies can be tested in isolation.



Explain Failures

Explains why a test failed and what to check first.



Suggest Refactoring

Proposes cleaner structure without changing behavior.



Require Human Review

Every suggestion needs your review before you trust it.

KEY TAKEAWAY Copilot handles the repetitive parts of testing — you still decide what's correct, secure, and worth shipping.

WHAT YOU'LL LEARN IN THIS MODULE

How these Copilot capabilities become skills you practice in this module.

WHAT YOU WILL LEARN



Leverage AI

Generate tests, cases, and mocks.



Improve Quality

Catch edge cases and gaps early.



Automate Repetitive Work

Focus your time on high-value review.



Enterprise Ready

Reliable, compliant testing at scale.

KEY TAKEAWAY These are the skills you'll build: leaning on AI, catching gaps early, automating the routine, and staying enterprise-ready.

GENERATING UNIT TESTS

GitHub Copilot helps you quickly generate meaningful unit tests for methods, classes, and services.

HOW TO GENERATE TESTS

- Select the method, class, or file you want to test.
- Use a prompt or comment asking Copilot to generate tests.
- Press Tab to accept, then review the generated assertions.
- Refine mocks, edge cases, and naming as needed.

EXAMPLE TEST (JUNIT 5 + MOCKITO)

```
@Test
void getUserById_existingUser_returnsUser() {
    User expected = new User(1L, "John");
    when(repository.findById(1L))
        .thenReturn(Optional.of(expected));
    User result = service.getUserById(1L);
    assertEquals("John", result.getName());
    verify(repository).findById(1L);
}
```

KEY TAKEAWAY Copilot accelerates unit test creation, but your understanding ensures the tests are correct and maintainable.

UNIT TEST BEST PRACTICES

Habits that keep Copilot-generated tests trustworthy and maintainable.

BEST PRACTICES



Test Behavior

What it does, not how.



Meaningful Names

Describe scenario + result.



Independent Tests

Each test runs in isolation.



Always Review

Validate before trusting it.

Try a specific prompt: "Generate JUnit 5 tests for CustomerService.updateStatus. Use Mockito for CustomerNotifier. Cover valid update, unknown customer, null status, duplicate status update, and notifier failure. Follow Arrange-Act-Assert and use descriptive test names."

KEY TAKEAWAY Behavior-focused names, independent tests, and a habit of review keep generated tests trustworthy.

TRY IT YOURSELF — EXERCISE 1: AAA TEMPLATE FOR STATUS

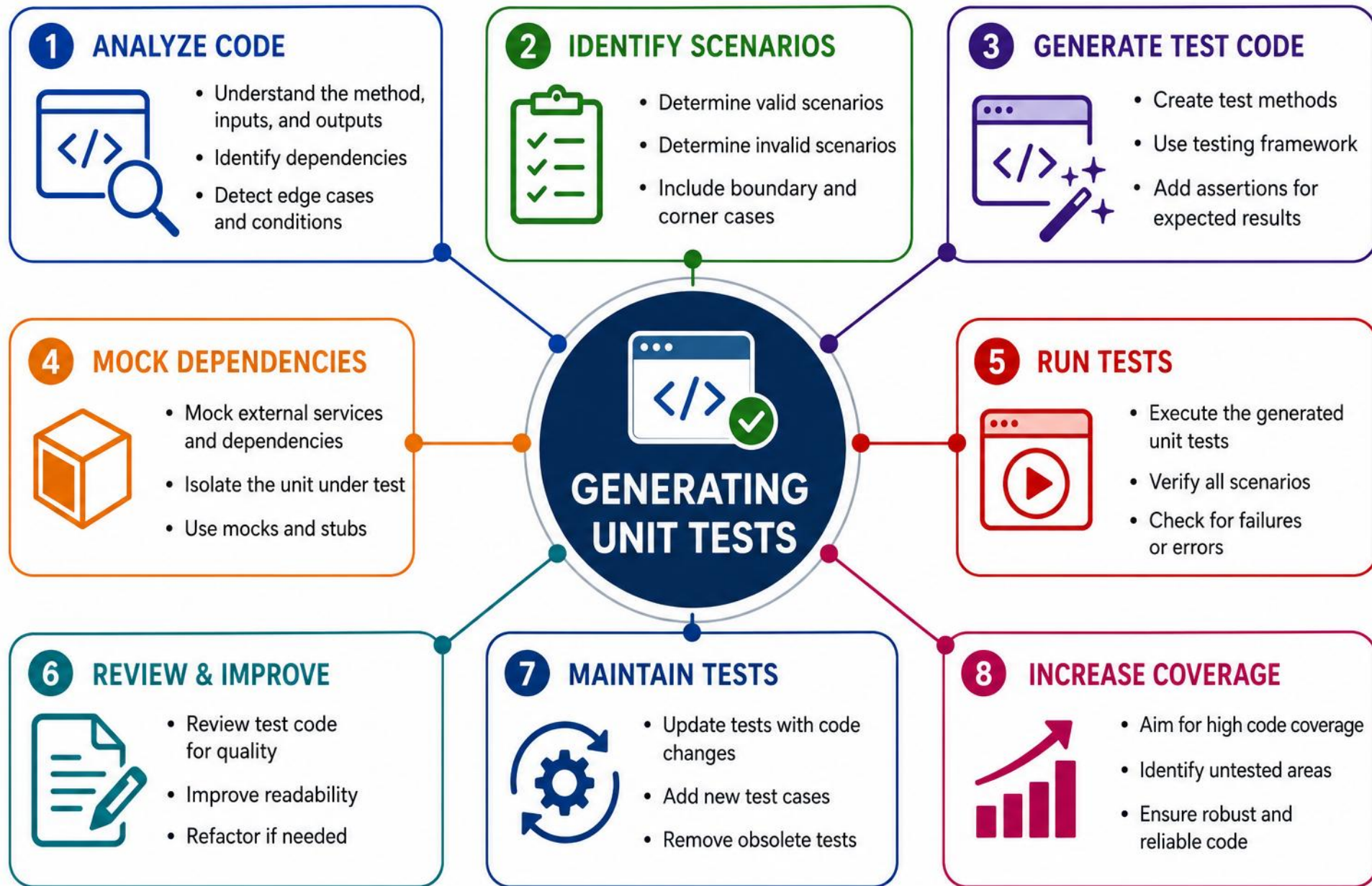
Reinforces: the Arrange-Act-Assert structure the prompt on the last slide asked Copilot to follow.

STEP	WHAT TO DO
Goal	Draft the AAA outline Copilot must follow for a Northstar customer status check
File	notes/lab11-prelab-aaa.md (in examples/module-11-exercises/)
Arrange	Build CUS-1002 Ravi as PROSPECT
Act	Call activate (conceptual) — leave blank here, fill in during Lab 11
Assert	Status becomes ACTIVE; correlation lab-request-001 logged later
Reference	exercises/exercise-01-aaa-template.md

```
# AAA outline -- Ravi PROSPECT -> ACTIVE
Arrange: build CUS-1002 Ravi as PROSPECT
Act:      <fill in during Lab 11>
Assert:   status == ACTIVE  // + lab-request-001 logged
activate_prospectRavi_setsStatusActive()  // name only -- Lab 18
```

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-aaa-template.md.





GENERATING EDGE CASES

Edge cases ensure your code is robust and works correctly under unusual or unexpected conditions.

CATEGORY	EDGE CASE	INPUT EXAMPLE	EXPECTED RESULT
Null	customerId is null	customerId = null	IllegalArgumentException
Empty	customerId is an empty string	customerId = ""	Validation failure
Missing customer	Customer ID not found	Unknown ID	CustomerNotFoundException
Null status	Status argument is null	status = null	Validation failure
Same status	Update repeats the existing status	Existing status repeated	No unnecessary notification
Notifier failure	Notifier throws while sending	Mock throws exception	Defined recovery or propagation
Duplicate customer	Same ID inserted twice	Same ID inserted twice	Reject duplicate

KEY TAKEAWAY Edge cases turn good code into great code — think broader, test deeper, deliver production-ready software.

EDGE CASE TESTS IN JUNIT 5

Copilot turns each row of the edge-case table into a concrete, runnable JUnit 5 test method.

FROM TABLE ROW TO TEST

- Null and empty inputs assert on the exception type Copilot suggested.
- Boundary and "not found" cases each get their own focused test.
- A repeated status update asserts that no notification is sent.
- Every generated test still needs a human-reviewed, meaningful assertion.

EXAMPLE TESTS (JUNIT 5)

```
@Test
void nullId_throwsIllegalArgumentException() {
    assertThrows(IllegalArgumentException.class,
        () -> service.activate(null, PROSPECT));
}

@Test
void unknownId_throwsNotFound() {
    String id = "CUS-9999";
    assertThrows(CustomerNotFoundException.class,
        () -> service.activate(id, PROSPECT));
}
```

KEY TAKEAWAY Turn each edge-case row into its own test — one behavior, one assertion, one clear name.

EDGE CASE TESTING BEST PRACTICES

Habits for finding and validating edge cases as code evolves.

BEST PRACTICES



Know Your Boundaries

Min, max, null, empty, invalid.



Think Like a User

How might they misuse it?



Happy & Sad Paths

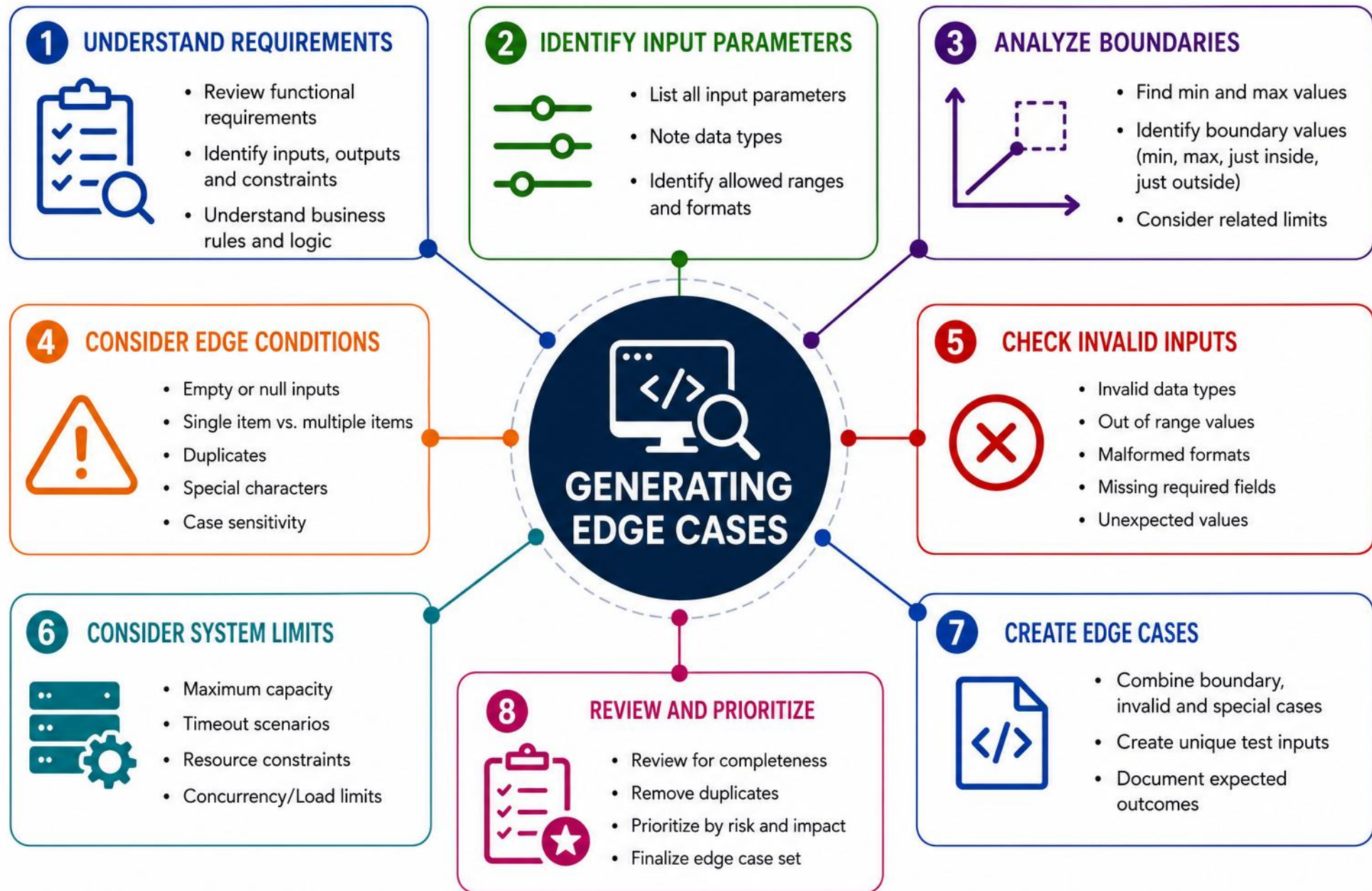
Success and failure scenarios.



Review & Refine

Add new edge cases as code evolves.

KEY TAKEAWAY Systematic edge-case habits — boundaries, misuse, both paths, and regular review — catch what happy-path testing misses.



GENERATING MOCK OBJECTS

GitHub Copilot helps you create mock objects and test doubles so you can test in isolation with predictable behavior.

WHAT COPILOT GENERATES

- Suggests the right mock objects for your dependencies.
- Generates stubs with default or custom return values.
- Sets up interactions — calls, arguments, order.
- Helps mock exceptions, timeouts, and failures.

EXAMPLE (MOCKITO)

```
@Mock
private PaymentGateway paymentGateway;

when(paymentGateway.charge(order))
    .thenReturn(new PaymentResult(true));
```

KEY TAKEAWAY Mock objects help you isolate units, control dependencies, and write fast, reliable tests — mock behavior, not implementation.

MOCK OBJECT BEST PRACTICES

Guidelines for mocking dependencies without over-specifying behavior.

BEST PRACTICES



Mock External Deps

Not the class under test.



Mock Behavior

Not internal state.



Stub Only What's Needed

Explicit return values.



Keep Tests Focused

One behavior per test.

Prefer fresh mocks and isolated test setup for each test. Avoid shared mutable state; use reset only in unusual cases.

Verify meaningful interactions only — don't mock value objects or simple domain classes.

Avoid over-specifying implementation details; prefer fakes when they produce clearer tests.

KEY TAKEAWAY Mock only what's external, stub only what's needed, and keep every test focused on one behavior.

VERIFYING MOCK INTERACTIONS

Stub a failure, then verify the interaction — proving the behavior a test claims to cover.

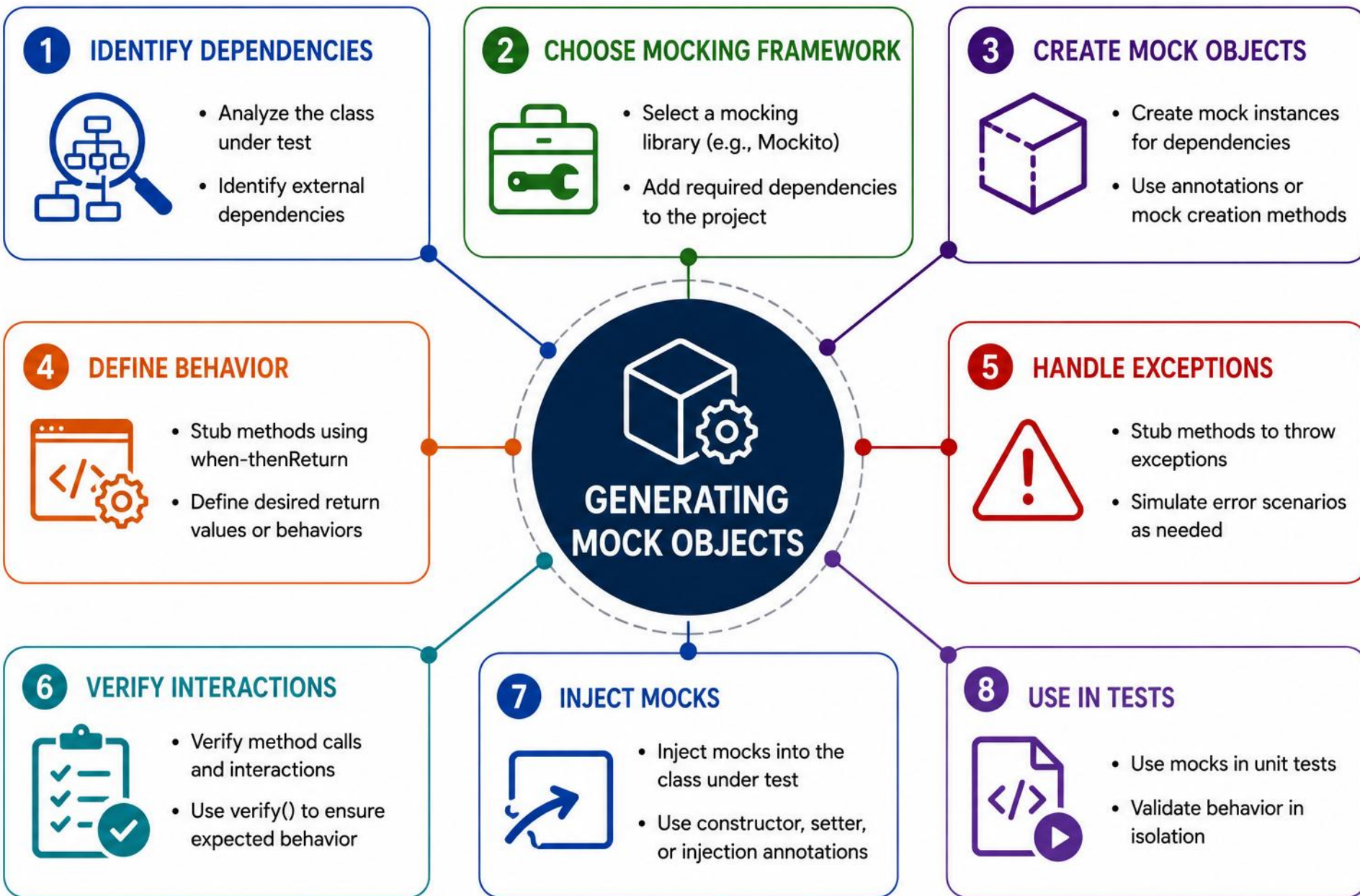
WHAT THIS ADDS

- Stub a dependency to throw, then assert the service recovers or propagates.
- `verify()` confirms a call actually happened — not just that the code compiled.
- `times(1)` and `never()` catch missing or duplicate calls.
- A fresh mock per test avoids shared, mutable state between tests.

EXAMPLE (MOCKITO)

```
@Test
void notifierFails_stillUpdatesStatus() {
    String id = "CUS-1002";
    doThrow(new RuntimeException("SMTP down"))
        .when(notifier).notifyActivated(id);
    Customer result = service.activate(id, ACTIVE);
    assertEquals(ACTIVE, result.getStatus());
    verify(notifier, times(1)).notifyActivated(id);
}
```

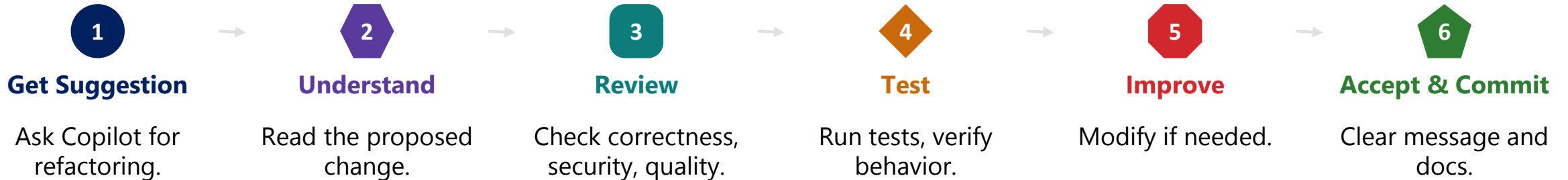
KEY TAKEAWAY `verify()` proves a call happened; a stubbed failure proves your code recovers — test both.



REFACTORING WITH AI

Copilot proposes refactorings, tests, and documentation — helping you improve code without changing behavior.

HOW COPILOT HELPS



KEY TAKEAWAY Refactor in small, tested steps — confirm behavior is unchanged before and after every change.

REFACTORING PROMPTS AND REMINDERS

Sample prompts and principles to guide AI-assisted refactoring.

EXAMPLE PROMPTS

- *"Refactor this method for readability without changing observable behavior. Preserve public signatures and existing exception behavior."*
- *"Analyze this method for performance bottlenecks. Do not change the code yet. Explain complexity, allocation hotspots, and what measurements would justify optimization."*
- *"Add null and empty checks with meaningful exceptions."*
- *"Suggest unit tests for this method using JUnit 5."*

ALSO REMEMBER

Refactor incrementally, write tests first, follow SOLID/DRY design principles, use version control, and measure impact (complexity, coverage, performance).

KEY TAKEAWAY Clear prompts and disciplined principles turn AI refactoring suggestions into safe, incremental improvements.

REFACTORING EXAMPLE: VALIDATION

Applying the null-check refactoring prompt from the previous slide to a real method.

WHAT CHANGED

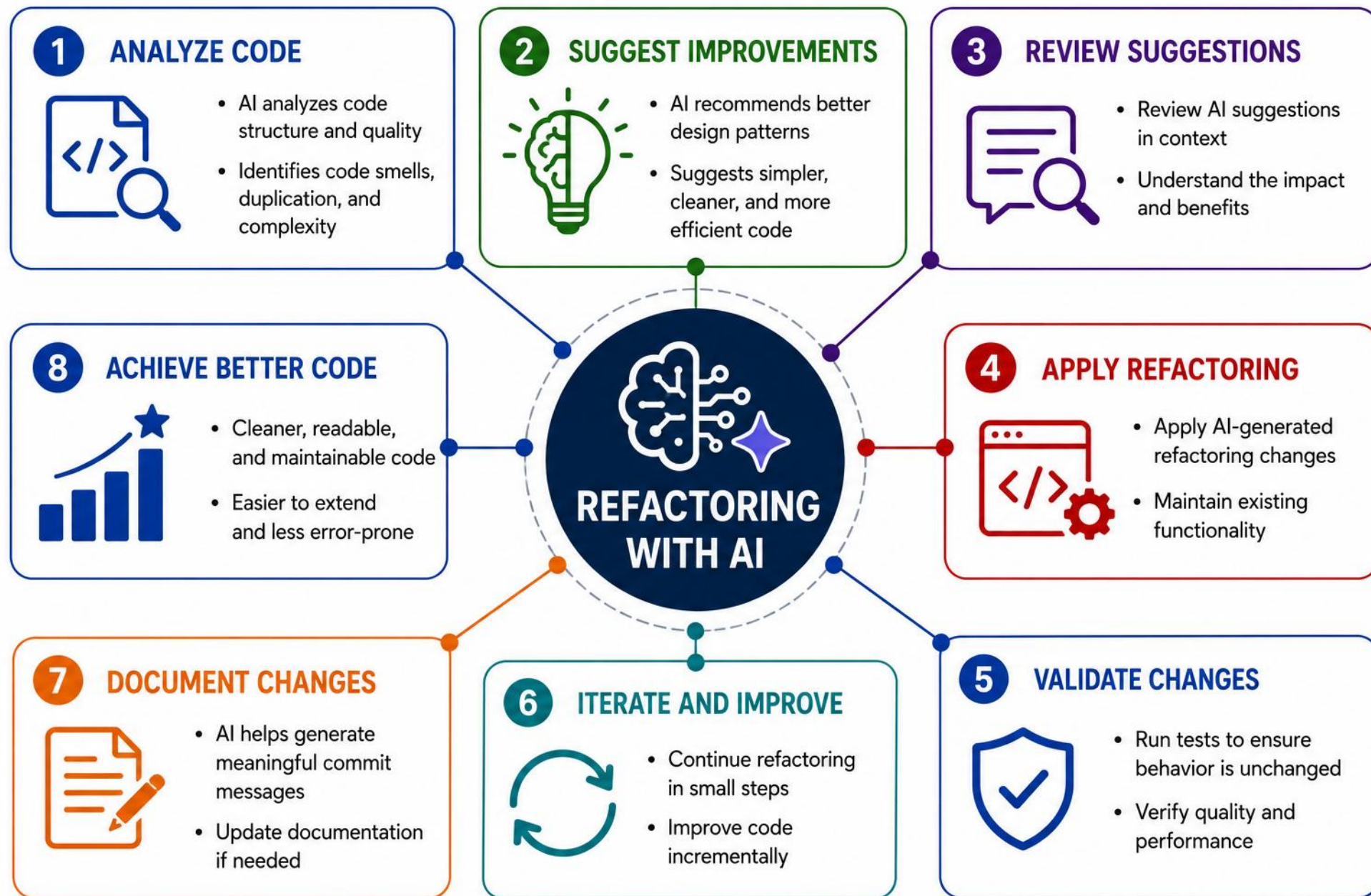
- The original method trusts its caller and fails deep inside with a `NullPointerException`.
- The refactored method validates first and throws a clear, typed exception.
- Behavior for valid input is unchanged — only invalid input behaves better.
- Existing tests still pass; a new test now covers the null-id case.

BEFORE / AFTER

```
// Before
Customer activate(String id, CustomerStatus s) {
    return find(id).setStatus(s);
}

// After
Customer activate(String id, CustomerStatus s) {
    if (id == null || id.isBlank()) {
        throw new IllegalArgumentException("bad id");
    }
    return find(id).setStatus(s);
}
```

KEY TAKEAWAY Validate first, fail with a typed exception, and keep the happy path unchanged.



DETECTING CODE SMELLS

GitHub Copilot helps you identify code smells early so you can reduce technical debt and improve maintainability. Treat size thresholds as prompts for review, not automatic proof of a smell — consider complexity, cohesion, domain clarity, and change frequency too.

CODE SMELL	WHAT IT MEANS	ILLUSTRATIVE EXAMPLE	COPILOT SUGGESTION
Long Method	Method does too many things	Method > 20-30 lines	Extract method, split responsibilities
Large Class	Too many fields/methods	Class > 15-20 methods	Extract class, apply SRP
Duplicated Code	Same code repeated	Similar blocks in many places	Extract method, reuse components
Long Parameter List	Too many parameters	> 4-5 parameters	Introduce Parameter Object
Magic Number/String	Hardcoded values	e.g. 0.90, "VIP"	Use constants, enums

KEY TAKEAWAY Code smells are warnings, not errors. Detect them early, refactor with confidence, and keep your codebase future-ready.

CODE SMELLS: BEFORE AND AFTER

Two code smells from the table, fixed with Extract Method and named constants.

WHAT COPILOT FLAGGED

- Magic Number: 0.90 and "VIP" hide a business rule with no name to search for.
- The tier check is duplicated logic that will drift as more tiers are added.
- Named constants and a small helper make both smells disappear.
- Behavior stays identical — only readability and intent improve.

BEFORE / AFTER

```
// Before
double price = 0.90 * base;
if (tier.equals("VIP")) price = 0.80 * base;

// After
static final double STANDARD_DISCOUNT = 0.90;
static final double VIP_DISCOUNT = 0.80;
double price = isVip(tier) ? base * VIP_DISCOUNT
                  : base * STANDARD_DISCOUNT;
```

KEY TAKEAWAY Extract Method and named constants turn a smelly one-liner into self-documenting code.

RESPONDING TO CODE SMELLS

A simple loop for acting on what Copilot flags.

HOW TO RESPOND



Detect Early

Review as you write code.



Understand the Smell

Read the explanation.



Refactor Incrementally

Small, safe changes.



Back with Tests

Ensure behavior stays the same.

KEY TAKEAWAY Detect smells early, understand why they matter, then refactor incrementally and back every change with tests.

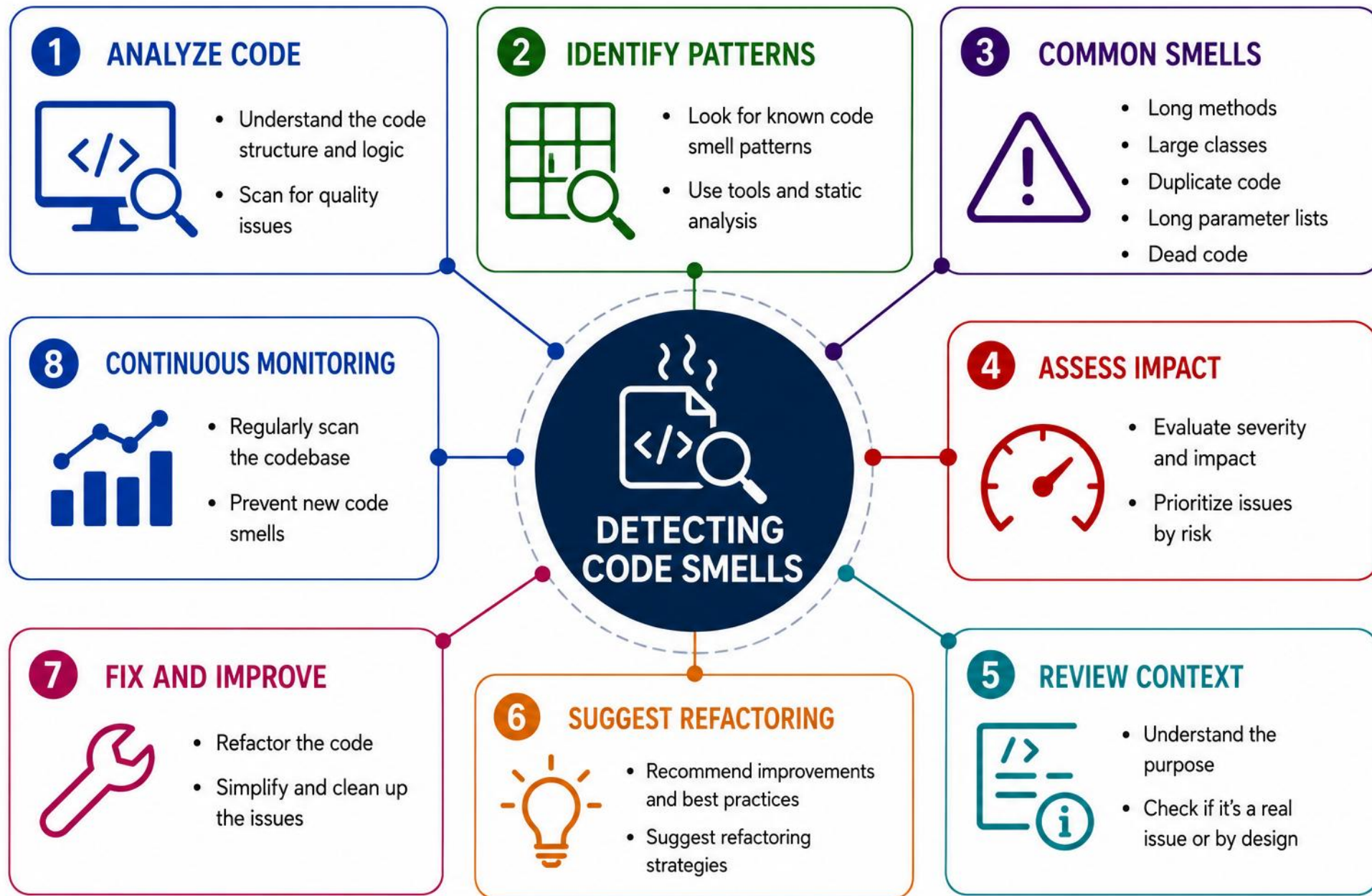
TRY IT YOURSELF — EXERCISE 2: NOTIFIER EXTRACT PLAN

Reinforces: naming a testability smell before you ask Copilot to refactor it.

STEP	WHAT TO DO
Goal	Plan extracting a notifier collaborator so tests can later isolate side effects
File	notes/lab11-notifier-extract-plan.md (in examples/module-11-exercises/)
Smell	Inline System.out / email send inside activate — a testability smell
Extract sketch	Interface CustomerNotifier.notifyActivated(customerId) — on paper only
Why for Copilot	Naming the collaborator stops AI from burying I/O inside the service
Reference	exercises/exercise-02-notifier-extract.md

```
// smell: I/O buried inside the service method
System.out.println("Activated " + id); // testability smell
emailClient.send(customer.getEmail(), ...); // hidden side effect
// extract sketch -- paper only, do not implement yet
interface CustomerNotifier { void notifyActivated(String id); }
```

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-notifier-extract.md.



REFACTORING QUALITY CHECKLIST

Copilot suggests clearer names, simpler logic, and smaller methods — see it applied to a real example.

HOW COPILOT HELPS

- Suggests clear, meaningful names for variables, functions, classes.
- Simplifies complex expressions and conditions into simpler steps.
- Extracts and organizes code into focused methods.
- Adds helpful comments that explain intent, not mechanics.

BEFORE / AFTER

```
// Before
double c(double p,double r,int y,int m){...}

// After
double computeCompoundInterest(
    double principal, double ratePercent) {...}
```

KEY TAKEAWAY Readable, well-structured code is a long-term investment — checked against a clear standard, not guesswork.

SIX THINGS TO VERIFY

A shared checklist for evaluating any Copilot-assisted refactor — from naming to behavior preservation.

QUALITY CHECKLIST: SIX THINGS TO VERIFY



Naming & Clarity

Names reveal intent without extra comments.



Method/Class Size

Size is a prompt to look closer, not an automatic rule.



Duplication

Extract shared logic instead of repeating it.



Coupling & Abstractions

Depend on interfaces, not another class's internals.



Domain Types

Use value objects and enums, not raw primitives.



Tests & Behavior Preservation

Existing tests still pass; behavior doesn't change.

KEY TAKEAWAY A refactor is only done when it passes every item on this checklist — not just when it compiles.

DOMAIN TYPES OVER PRIMITIVES

Illustrating the Domain Types checklist item — an enum expresses intent a raw String cannot.

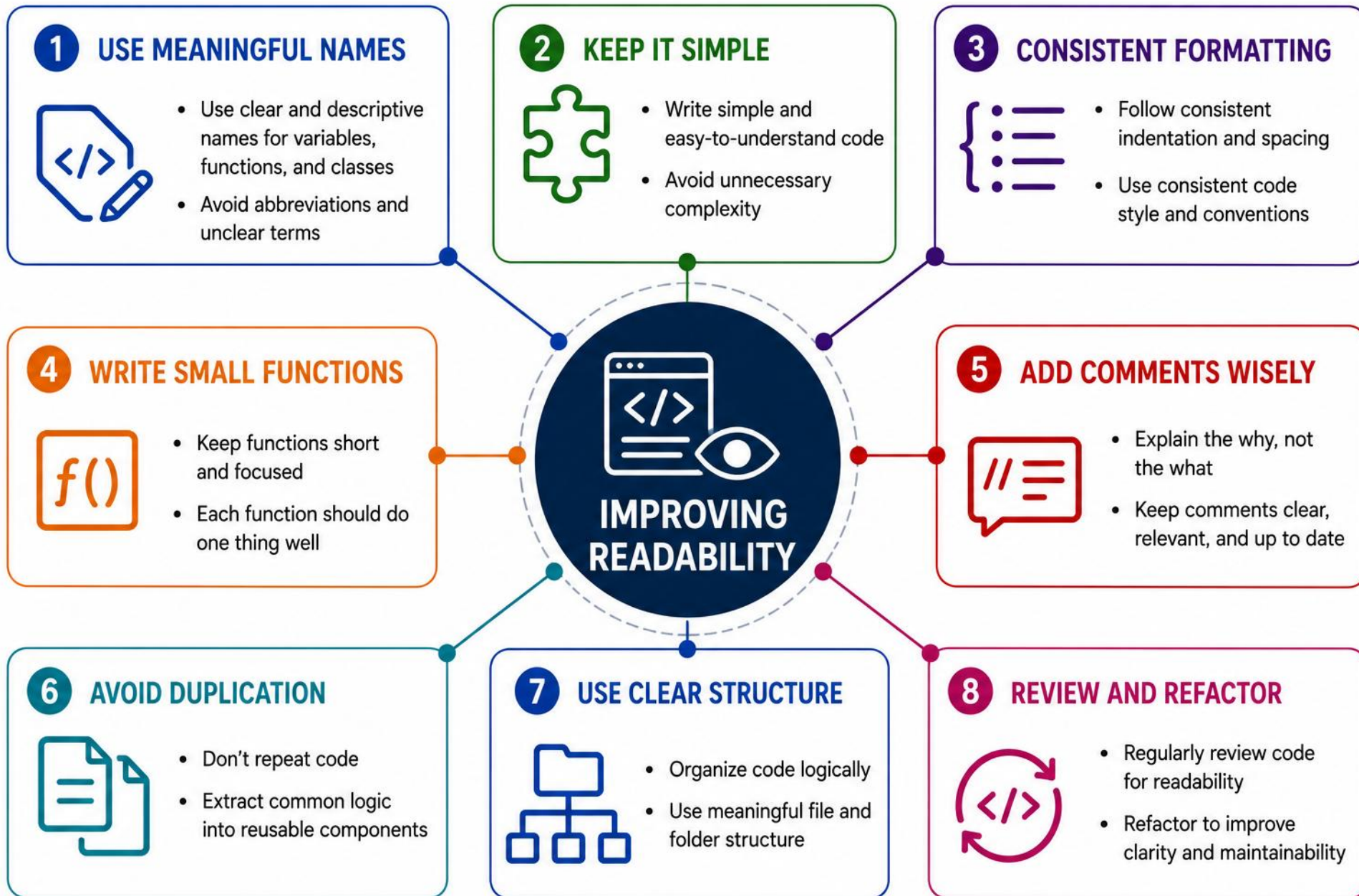
WHY IT MATTERS

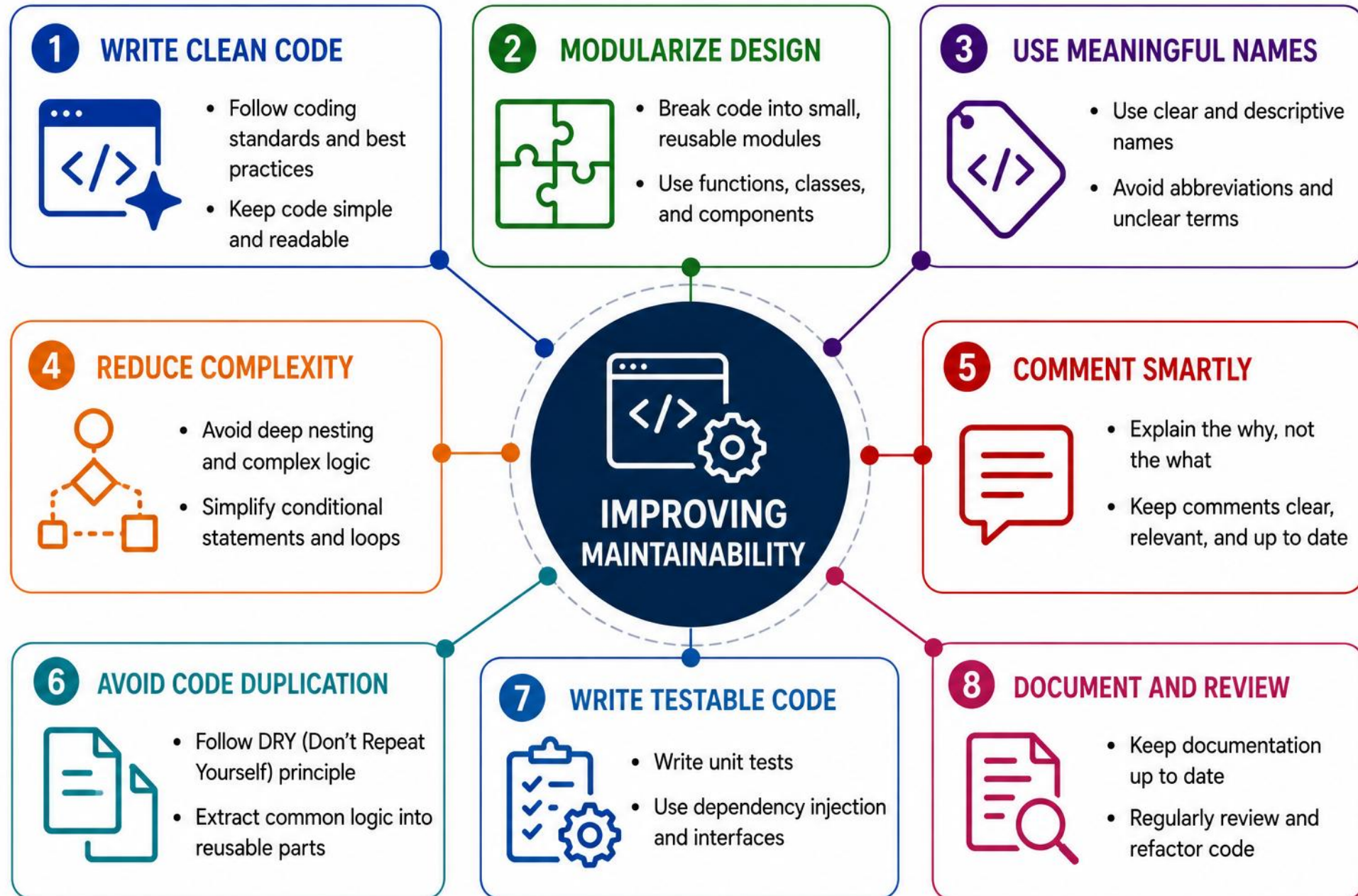
- A raw String status accepts anything: "ACTIVE", "active", typos — the compiler won't catch it.
- An enum makes illegal states unrepresentable and lists the valid set in one place.
- Switches over an enum can be checked for missing cases by tools and reviewers.
- Copilot suggests the enum once it sees status compared against string literals.

EXAMPLE (ENUM)

```
public enum CustomerStatus {  
    PROSPECT, ACTIVE, SUSPENDED, CLOSED  
}  
  
public Customer activate(String id,  
    CustomerStatus status) {  
    Customer c = find(id);  
    c.setStatus(status);  
    return c;  
}
```

KEY TAKEAWAY Enums make invalid states impossible to represent — the compiler enforces what comments cannot.





REVIEWING AI-GENERATED TESTS AND REFACTORS

AI can generate great tests and refactors — but you are the final reviewer of everything it produces.

WHY YOU MUST REVIEW



Correctness

Works as intended.



Security

No vulnerabilities.



Quality

Consistent with standards.



Maintainability

Easy to read and test.



Confidence

Improves your team.

KEY TAKEAWAY AI is your assistant, not a replacement — review its tests and refactors as carefully as you would your own.

REVIEW CHECKLISTS: TESTS VS REFACTORS

Different questions apply to AI-generated tests than to AI-assisted refactors — here's what to check for each.

FOR AI-GENERATED TESTS

- Fails when the production code is broken?
- Assertions are meaningful, not just non-null checks?
- Tests are independent of each other?
- Mocks aren't hiding real behavior?
- Negative and boundary cases are covered?
- Verifies public behavior, not internals?

FOR AI-ASSISTED REFACTORS

- Observable behavior is preserved?
- All existing tests still pass?
- Complexity has actually improved?
- Public APIs are unchanged?
- Exceptions, logging, and transactions are unchanged?
- The diff is smaller and easier to understand?

GOLDEN RULE AI writes the first draft. You ensure it's production-ready.

KEY TAKEAWAY AI writes the first draft. You ensure it's production-ready — different checks for tests than for refactors.

TRY IT YOURSELF — EXERCISE 3: TRIVIAL VS REAL ASSERTS

Reinforces: assertions must be meaningful, not just non-null checks.

STEP	WHAT TO DO
Goal	Label weak Copilot asserts vs ones that protect Amina/Ravi's real behavior
File	notes/lab11-prelab-asserts.md (in examples/module-11-exercises/)
Trivial	assertNotNull(customer); assertTrue(true) — these can never fail
Meaningful	Amina (CUS-1001) status equals ACTIVE; Ravi (CUS-1002) equals PROSPECT
Reject rule	Reject any AI test whose asserts never mention a domain value or outcome
Reference	exercises/exercise-03-trivial-vs-real-asserts.md

```
// trivial -- can never catch a real regression
assertNotNull(customer); assertTrue(true);
// meaningful -- protects Amina/Ravi's actual behavior
assertEquals(ACTIVE, find("CUS-1001").getStatus()); // Amina
assertEquals(PROSPECT, find("CUS-1002").getStatus()); // Ravi
```

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-trivial-vs-real-asserts.md.

TRY IT YOURSELF — EXERCISE 4: ACCEPTANCE GATE (STARTER)

Reinforces: gate every Copilot test with the same checklist you just practiced — fill-in-the-blank, not from scratch.

STEP	WHAT TO DO
Goal	FILL-IN-THE-BLANK STARTER — replace every ____ below for a sample AI-generated test
File	notes/lab11-acceptance-todos.md (in examples/module-11-exercises/)
Fill in	6 blanks: AAA present, Amina/Ravi asserts, trivial rejected, correlation, gap, Accept/Reject
Gap sentence	"Deep Mockito verify waits for Lab 18; JaCoCo gates deepen in Lab 17."
Self-check	Confirm the Labs 17–18 boundary sentence appears in your filled sheet
Reference	exercises/exercise-04-fill-acceptance-checklist-todos.md (starter)

AAA structure present? _____

Asserts use Amina/Ravi values? _____

Trivial assertNotNull-only rejected? _____

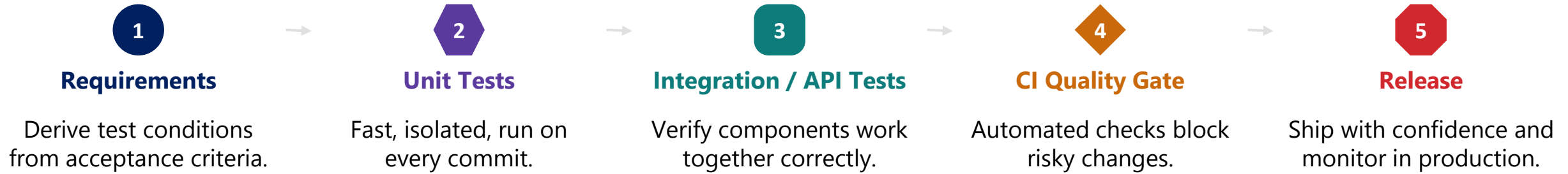
Correlation mention lab-request-001? _____

Coverage gap (Labs 17-18)? _____ Accept/Reject: _____

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-fill-acceptance-checklist-todos.md (starter).

WHERE AI-ASSISTED UNIT TESTING FITS

A repeatable flow from requirements to release, with quality gates along the way. Toolchain examples: Docker/Kubernetes, Jira/Azure DevOps, Selenium/Postman/JUnit, JMeter/k6/OWASP ZAP, Datadog/Grafana/Allure — versions per the course setup guide.



KEY TAKEAWAY Track pass rate, coverage trend, and flaky/escape rates together — no single metric proves quality on its own.

CORE QUALITY METRICS

The numbers that show whether your test suite is actually working.

CORE QUALITY METRICS



Test Pass Rate

Share of tests passing on the latest build.



Coverage (as a Signal)

Shows what ran, not what was verified — track the trend.



Mutation Score (if applicable)

How many injected defects the tests actually catch.



Flaky-Test Rate

Tests that pass or fail without any code change.



Defect Escape Rate

Bugs found in production that tests should have caught.



Test Execution Time

How long the suite takes — keep feedback loops fast.

KEY TAKEAWAY No single metric proves quality — track pass rate, coverage trend, flakiness, and escapes together.

TRY IT YOURSELF — EXERCISE 5: COVERAGE GAPS MAP

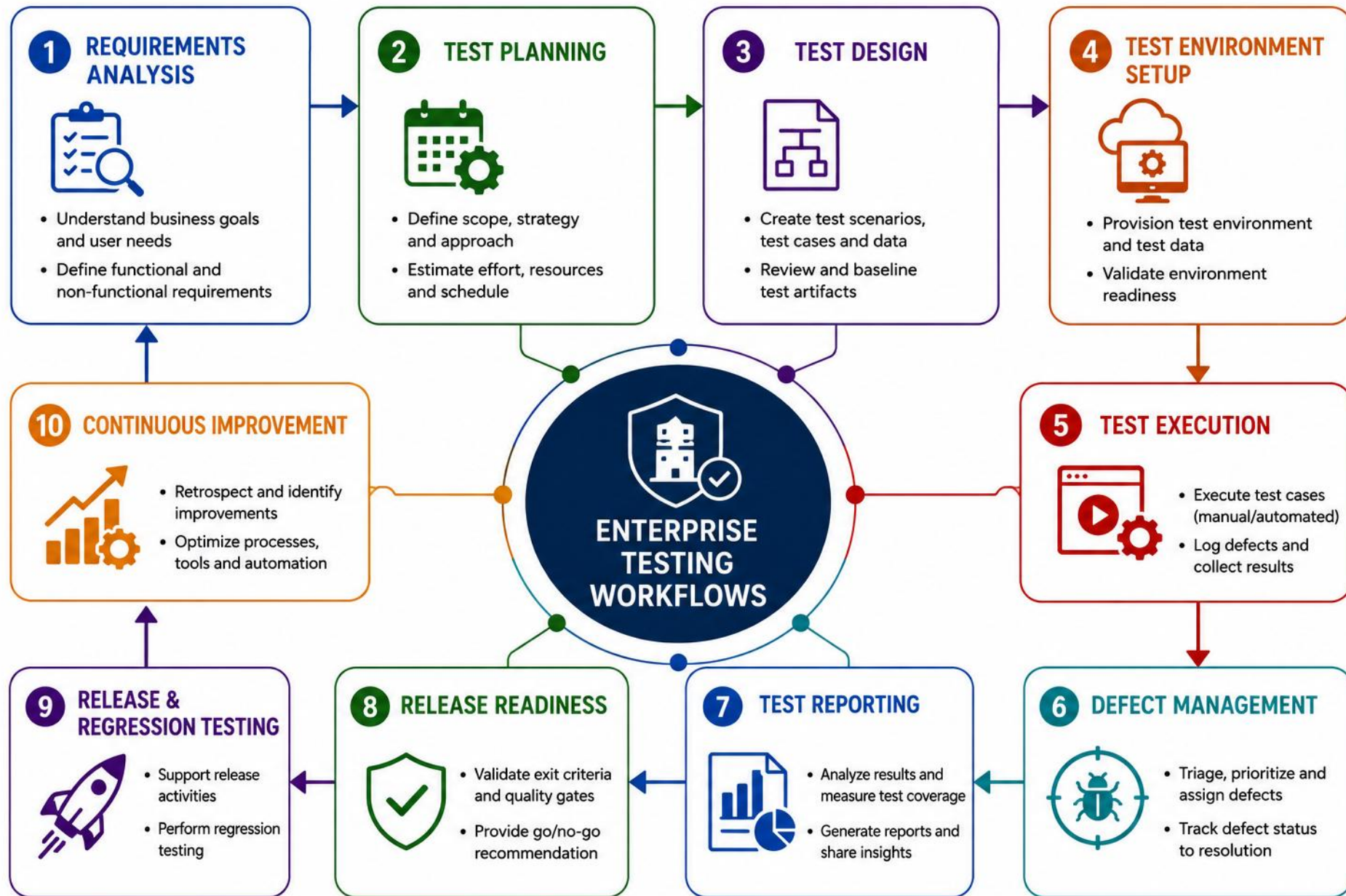
Reinforces: coverage shows what ran, not what Labs 17–18 will still deepen.

STEP	WHAT TO DO
Goal	Name what Lab 11 prep will not prove versus later testing labs
File	notes/lab11-coverage-gaps.md (in examples/module-11-exercises/)
In scope now	AI-assisted happy-path test sketch, AAA discipline, reject trivial asserts
Lab 17	Parameterized tests, naming conventions, and the JaCoCo narrative deepen here
Lab 18	Stub vs verify and ArgumentCaptor — deep Mockito isolation waits for Lab 18
Reference	exercises/exercise-05-coverage-gaps.md

```
# Coverage gap map -- Lab 11 prep vs later labs
In scope now:  happy-path sketch, AAA discipline,
                reject trivial asserts

Lab 17:        parameterized tests, naming, JaCoCo narrative
Lab 18:        stub vs verify, ArgumentCaptor, deep isolation
```

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-coverage-gaps.md.



TRY IT YOURSELF — EXERCISE 6: LAB 11 PREP CHECKLIST

Reinforces: confirm you're ready to practice AI test generation — before Lab 11 begins, not instead of it.

STEP	WHAT TO DO
Goal	Confirm you're ready to practice AI test generation without finishing Lab 11 now
File	notes/lab11-prep-checklist.md (in examples/module-11-exercises/)
Fixtures	From memory: Amina = CUS-1001 ACTIVE; Ravi = CUS-1002 PROSPECT
Checklist present	Confirm notes/lab11-acceptance-todos.md exists from Exercise 4
Boundary	"Pre-lab only — prepare for the lab; do not complete full Lab 11 now."
Reference	exercises/exercise-06-lab11-prep-checklist.md

```
Pass if all three notes files exist:  
[ ] AAA template           (Exercise 2)  
[ ] Acceptance TODOs       (Exercise 4)  
[ ] Coverage gap map       (Exercise 5)  
Else: revisit Exercises 2, 4, and 5 before Lab 11.
```

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-lab11-prep-checklist.md.

LAB OVERVIEW – AI-ASSISTED TEST GENERATION

You extend the Northstar CRM's CustomerService with a safety-net test suite and a cleaner design.

SAMPLE CODE UNDER TEST

```
public class CustomerService {  
    private final List<Customer> customers;  
    private final CustomerNotifier notifier;  
    public Customer updateStatus(  
        String id, CustomerStatus s) {...}  
}
```

LAB WORKFLOW

- Copy Lab 10's CustomerService; add JUnit 5 + Mockito.
- Generate tests with Copilot; reject weak assertions.
- Extract CustomerNotifier; verify it with a mock.
- Review coverage gaps; write acceptance guidelines.

KEY TAKEAWAY Estimated time: 45 minutes. Build the safety net first, then refactor with confidence.

LAB OBJECTIVES AND TOOLS

What you'll practice, and what you'll need.

LAB OBJECTIVES



Generate Unit Tests

For Customer and CustomerService.



Reject Weak Tests

Spot assertions that can't fail.



Extract & Mock

CustomerNotifier, verified with Mockito.



Review & Refine

Coverage gaps, acceptance guidelines.

Tools: GitHub Copilot, IntelliJ IDEA or VS Code, JDK 21+, Maven, JUnit 5, Mockito — use the versions supplied by the starter POM or the current course setup guide.

KEY TAKEAWAY Know your objectives and tools before you start — then build the safety net first.

LAB TASKS: BUILD

Write the safety net and extract the dependency.

#	TASK	DETAILS	FORMAT
1	Entity & Service Tests	CustomerTest and CustomerServiceTest via Copilot.	.java
2	Reject False Confidence	Catch and replace an assertion that can't fail.	.java
3	Extract & Mock Notifier	CustomerNotifier interface, verified with Mockito.	.java
4	Refactor Duplication	Extract validateCustomerId(); confirm with tests.	.java

KEY TAKEAWAY Start by building the safety net — tests, false-confidence checks, and a clean extraction — before you verify and document.

LAB TASKS: VERIFY & DOCUMENT

Confirm coverage, document guidelines, and validate with mutation testing.

#	TASK	DETAILS	FORMAT
5	Coverage Gap Review	List covered/untested methods; note honest gaps.	.md
6	Acceptance Guidelines	5-point checklist for AI-generated tests/refactors.	.md
7	Mutation Check (Optional)	Break a production result, confirm the test fails, then restore it and record what the test caught.	.md

ACCEPTANCE CRITERIA

- All ~7 tests pass with ``mvn clean test``; CustomerNotifier is extracted and verified via Mockito; at least one false-confidence test is rejected and documented; coverage gaps are documented honestly, not chased to 100% — coverage shows which code executed, not whether behavior was verified correctly.

KEY TAKEAWAY Ship code you understand and can defend — tests, mocks, and refactors included.

AI-ASSISTED TESTING: DEFINITION OF DONE

Before you call a test suite or refactor finished, check it against this list.

- 1 Tests verify public behavior, not internal implementation.
- 2 Assertions can actually fail — no tautologies.
- 3 Happy, negative, and boundary paths are covered.
- 4 Mocks isolate only true external dependencies.
- 5 Tests are deterministic and independent of each other.
- 6 Refactoring preserves observable behavior.
- 7 Existing and new tests all pass.
- 8 Coverage gaps are documented honestly, not hidden.
- 9 AI suggestions are understood and reviewed before accepting.
- 10 Final code meets team standards and conventions.

KEY TAKEAWAY A test suite is done when it verifies behavior, fails honestly, and you understand every AI-assisted line in it.

KNOWLEDGE CHECK (1 OF 2)

Test your understanding of AI-assisted testing and refactoring concepts.

1. What's the primary benefit of AI for test generation?

- A. It replaces human testers completely
- B. It increases test coverage and speed**
- C. It eliminates the need for unit tests
- D. It removes the need for code reviews

3. What's the key advantage of testing in a CI pipeline?

- A. Manual test execution
- B. Early feedback and continuous validation**
- C. Reduced need for edge cases
- D. Elimination of unit tests

2. Which tool creates mocks for external dependencies?

- A. JUnit
- B. Mockito**
- C. JaCoCo
- D. SonarQube

4. Which AI-generated test creates false confidence?

- A. A test with a meaningful expected exception
- B. A test that only asserts `result != null`**
- C. A test verifying repository interaction
- D. A parameterized boundary test

KEY TAKEAWAY AI accelerates test generation — coverage and speed improve, but human review still ensures quality.

KNOWLEDGE CHECK (2 OF 2)

Four more questions, plus the full answer key.

5. What's the goal of the AAA pattern in unit tests?

- A. Arrange, Act, Assert**
- B. Analyze, Act, Adjust
- C. Automate, Analyze, Assess
- D. Arrange, Automate, Assert

7. Why are edge cases important in testing?

- A. They test the happy path
- B. They help uncover hidden defects**
- C. They validate security policies
- D. They reduce the number of test cases

ANSWER KEY

• 1-B 2-B 3-B 4-B 5-A 6-B 7-B 8-C

6. Which is a best practice for AI-assisted testing?

- A. Use AI output without any changes
- B. Keep tests independent and atomic**
- C. Test only the happy path
- D. Avoid writing unit tests manually

8. What should you do with AI-generated tests?

- A. Run them without any changes
- B. Accept them if they compile
- C. Review, refine, and validate them**
- D. Use them only in production

KEY TAKEAWAY AI accelerates testing — higher quality, more productivity, greater confidence — but review, refine, validate is still yours.

MODULE 11 COMPLETE

GitHub Copilot for Testing and Refactoring

You can now use AI to generate tests, uncover edge cases, and refactor code — while staying the final, critical reviewer.